

Trajectory Prediction of a Pegasus XL Sounding Rocket for Vertical Payload Deployment

C. Emmett, M. Forte, K. Graziose, K.C. Madden, B.C. Tiamson

Team Name, Newark, Delaware 19716, United States

I. Abstract

The abstract should appear at the beginning of your paper. It should be one paragraph long (not an introduction) and complete in itself (no reference numbers). It should indicate subjects dealt with in the paper and state the objectives of the investigation. Newly observed facts and conclusions of the experiment or argument discussed in the paper must be stated in summary form; readers should not have to read the paper to understand the abstract. The abstract should be bold, indented 3 picas (1/2") on each side, and separated from the rest of the document by - blank lines above and below the abstract text.

II. Nomenclature

A	= vehicle cross-sectional area
c	= effective exhaust velocity
$\bar{c}$	= average launch effective exhaust velocity
C_D	= drag coefficient
g	= gravity
$\bar{g}$	= average launch gravity
g_0	= initial gravity
m_0	= initial mass
R_0	= initial radius from the center of Earth
t_p	= time at power cut-off
t	= time
v	= vehicle velocity
h	= altitude (height)
ρ	= mass density of the atmosphere
ζ	= propellant mass fraction

III. Introduction

The main objective of this study is to analyze the performance of a Northrop Grumman Pegasus XL Launch Vehicle used as a sounding rocket to launch a payload of 3000 kilograms in a vertical trajectory. Though the Pegasus is typically launched from a carrier aircraft, it has been reevaluated as a three-stage sounding rocket for this study. The team has developed code for a one-dimensional trajectory accounting for factors such as gravity, drag, and the effects of the changing atmosphere as the vehicle ascends, to calculate the maximum altitude of the launch vehicle. Several assumptions have been made to simplify these calculations, detailed in the Methods section. The trajectory has been designed for the following motors, which will define the three stages of launch: (1) Orion 50S XL; (2) Orion 50 XL; (3)

Orion 38. The code uses data from the motor specifications for these motors according to the provided ATK Motor Catalog. This report outlines the methods, results analysis, and conclusions of this study, using fundamental rocket propulsion concepts and elementary flight mechanics for multi-stage vehicles to determine the maximum altitude achieved by the launch vehicle.

IV. Methods

Assumptions

To simplify mathematical calculations, some major assumptions were made in this analysis. The section below lists them all:

1. The first assumption is that thrust is a constant for each stage according to the Orbital ATK Motor Catalog, from a time average. In reality, thrust comes from a vacuum thrust vs time curve, and nozzle exit pressure would have to be determined.
2. The second assumption is instantaneous stage separation and subsequent stage ignition. Again, in reality, this would not be possible as there is a moment between separation, while the stage falls away, and ignition. This is a brief period of coasting and is done also to save a small amount of propellant. This analysis assumes they happen at the same time as that timing would be hard to estimate.
3. Figure 4-4 from the textbook referenced shows the coefficient of drag versus Mach number for a range from 0.2 to 5.4 for a German V-2 Rocket at vertical flight. This analysis uses an interpolation in accordance with that graph and assumes anything above the Mach number of 5.4 to have a coefficient of drag of 0.15, the maximum value from the text [1].
4. This analysis also assumes a perfectly vertical, sounding, trajectory. This means all motion accounted for is in the perfect vertical direction. Without this assumption, gravity and coefficient of drag would also be functions of the angle of attack, something far too complicated for the scope of this course.
5. Temperature and air density are given by a function from NASA. The function considers one relationship for altitudes below 11,000 meters, another between 11,000 and 25,000, and another for anything higher than that [2].

Dynamical Equation

The equation below governs the dynamics of the system.

$$\begin{aligned} dv/dt &= a(v, h, t) \\ &= \frac{c(h) \cdot \zeta/t_p}{1 - \zeta \cdot t/t_p} - g(h) - \frac{1/2 \cdot C_D(v) \cdot \rho(h) \cdot v^2 A/m_o}{1 - \zeta \cdot t/t_p}, \end{aligned} \quad [X]$$

which follows the nomenclature as provided in Section II. c , g , and ρ are a function of the altitude, h , and C_D is a function of Mach number, though it is denoted as a function of velocity, v , above.

Numerical Method

The numerical method chosen to integrate the dynamical equation above is called Euler's method. It was chosen for its simplicity as it is a first-order method. Euler's method is defined by Equation [X] below

$$y_{n+1} = y_n + h y'_n, \quad [X]$$

where y is some function, y' is the first derivative of that function, h is the size of the timestep, and n denotes which timestep. This method can only solve first-order differential equations. The governing dynamical equation is a second-order differential equation, so it must be reduced into a system of two first-order differential equations. This system is shown below.

$$\begin{aligned} v_{n+1} &= v_n + a_n \cdot \Delta t, \\ h_{n+1} &= h_n + v_n \cdot \Delta t. \end{aligned} \quad [X]$$

The variables a , v , h , represent the acceleration, velocity, and altitude of the rocket at timestep n .

Important Functions

Three functions are required to be defined in order to find the acceleration, shown below. The first is a function to find the gravitational acceleration at a given height, based on the Universal Gravitational constant, radius of the Earth, height above the surface, and mass of the earth.

$$g(h) = \frac{G * M_{earth}}{(R_0 + h)^2} \quad [X]$$

Second is a function for temperature given the height, which is based on the NASA relation described earlier. Next is the density at a given altitude as well, coming from that same source, which uses another function for pressure.

$$T(h) = 288.19 - 0.00649h \quad (h < 11,000) \quad [x]$$

$$P(h) = 101.29 * \left(\frac{T}{288.08} \right)^{5.256}$$

$$T(h) = 216.69 \quad (11,000 < h < 25,000)$$

$$P(h) = 22.65 * e^{1.73 - .000157h} \quad [x]$$

$$T(h) = 141.94 + 0.00299h \quad (h > 25,000)$$

$$P(h) = 2.488 * \left(\frac{T}{216.6} \right)^{-11.388} \quad [x]$$

$$\rho(h) = \frac{P}{0.2869T} \quad [x]$$

Finally, a function for the Mach number was needed, as it changes with both temperature and vehicle speed. This function assumed a constant specific heat ratio throughout the flight and is given below:

$$Mach(T, u) = \frac{u}{\sqrt{\gamma \frac{R}{MW_{air}} T}} \quad [x]$$

Implementation

V. Results and Analysis

VI. Conclusion

Appendix

Appendix A: Main Code

```
function [Altitude, Velocity, Time] =  
SingleStageRocket(vInit,tInit,hInit,Thrust,Zeta,cD,mO,Area,tP,tAfter)  
  
%Inputs  
    %vInit - Initial velocity, required for intermediate stages of the %rocket  
    %tInit- Initial time, required for intermediate stages of the rocket  
    %hInit - Initial altitude, require for intermediate stages of the %rocket  
    %Thrust - Dependent on motor  
    %Zeta - Dependent on Motor, function of mass ratio  
    %cD - Drag Coefficient, may dependent on stage?  
    % mO - initial mass  
    %Area - cross sectional contact area (drag)  
    %tP - Dependent on motor  
    %tAfter- Next stage may take some time to start up, can also s  
  
%Initializing Variables  
    %Time  
    dt = 0.001; %timestep size - smaller means greater accuracy  
    tTotal = tP + tAfter;  
    timesteps = tTotal/dt;  
  
    %Outputs  
    Altitude(1:timesteps) = 0;  
    Velocity(1:timesteps) = 0;  
    Time(1:timesteps) = 0;  
  
    %Initial Conditions  
    Altitude(1)=hInit;  
    Velocity(1)=vInit;  
    Time(1)=tInit;  
  
    %Required Functions  
    %Gravity  
    function gVal = g(h)  
        g0=0; %m/s  
        R0=6378*10^3; %meters  
        gVal = ((R0/R0+h)^2)*g0;  
    end  
  
    %Effective Exhaust Velocity  
    function cVal = c(h)  
        cVal=Thrust; %replace with function for effective exhaust velocity vs height
```

```

        end

        %Air Density
        function rhoVal = rho(h)
            rhoVal=1.29; %replace with function of air density vs height.
        end

        %Approach: Split second order ODE to system of 1st order ODE, use Euler's method to numerically
        integrate
        %System:  $dh/dt = u$ ,  $du/dt = F/m = a$ 

        %Eulers Method
        for i = 1:timesteps-1
            a = (c(Altitude(i))*(Zeta/tP))/(1-Zeta*((Time(i)-tInit)/tP)) - g(Altitude(i)) -
            (cD*0.5*rho(Altitude(i))*Velocity(i)^2*Area/mO)/(1-Zeta*((Time(i)-tInit)/tP));%defining  $du/dt = a$ 

            Velocity(i+1) = Velocity(i) + a*dt;

            Altitude(i+1) = Altitude(i) + Velocity(i) * dt;

            Time(i+1)=Time(i)+dt;
        end

    End

    import numpy as np
    import matplotlib.pyplot as plt

    '''
    This code is used to predict the trajectory of a 3-Stage Pegasus XL ground
    launched vehicle in strict vertical flight, subjected to gravity and drag
    in the atmosphere.

    First Stage: Orion 50S XL
    Second Stage: Orion 50 XL
    Third Stage: Orion 38

    The following assumptions have been made to simplify the computation:
    - Constant average thrust for each stage is used from the Orbital ATK
    Motor Catalog
    - Instant stage separation and next stage ignition
    - Drag coefficient is approximated versus Mach # for a range from 0.2 to
    5.4. Anything above 5.4 is estimated as 0.15
    '''

```

```

##### Stage Parameters #####
##### First Stage: Orion 50S XL #####
s1_tp = 69.10 # stage 1 burn time (69.1s)
s1_impulse = 4.331E7 # stage 1 total impulse (N-sec)
s1_avgT = 6.263E5 # stage 1 burn time average thrust (N)
s1_massTotal = 16173 # stage 1 total mass (kg)
s1_massProp = 15023 # stage 1 propellant mass (kg)
s1_massBurn = 1092 # stage 1 burnout mass (kg)
s1_diam = 1.270 # stage 1 motor diameter (m)

s1_sep = s1_tp # time of stage 1 separation
s1_c = s1_avgT * s1_tp / s1_massProp # stage 1 exhaust velo
s1_mDot = s1_massProp / s1_tp # stage 1 mass flow rate

##### Second Stage: Orion 50 XL #####
s2_tp = 69.70 # stage 2 burn time (69.7s)
s2_impulse = 1.12E7 # stage 2 total impulse (N-sec)
s2_avgT = 1.606E5 # stage 2 burn time average thrust (N)
s2_massTotal = 4318 # stage 2 total mass (kg)
s2_massProp = 3923.6 # stage 2 propellant mass (kg)
s2_massBurn = 373.76 # stage 2 burnout mass (kg)
s2_diam = 1.270 # stage 2 motor diameter (m)

s2_sep = s2_tp + s1_sep # time of stage 2 separation
s2_c = s2_avgT * s2_tp / s2_massProp # stage 2 exhaust velo
s2_mDot = s2_massProp / s2_tp # stage 2 mass flow rate

##### Third Stage: Orion 38 #####
s3_tp = 67.70 # stage 3 burn time (67.7s)
s3_impulse = 2.184E6 # stage 3 total impulse (N-sec)
s3_avgT = 3.223E4 # stage 3 burn time average thrust (N)
s3_massTotal = 891.76 # stage 3 total mass (kg)
s3_massProp = 770.65 # stage 3 propellant mass (kg)
s3_massBurn = 110.22 # stage 3 burnout mass (kg)
s3_diam = 0.9652 # stage 3 motor diameter (m)

s3_sep = s3_tp + s2_sep # time of stage 3 separation
s3_c = s3_avgT * s3_tp / s3_massProp # stage 3 exhaust velo
s3_mDot = s3_massProp / s3_tp # stage 3 mass flow rate

##### Payload #####
pl_mass = 3000 # payload mass (kg)
#####
'''

```

General Method

1. Iterate over timesteps

2. Calculate acceleration from force balance (forces change with time and height)
3. Use velocity and timestep to calculate height

Timeline

Stage 1 -> Stage 2 -> Stage 3 -> Burnout

$du/dt = a = sX_avgT / \text{total mass} - g - 0.5 * cd * rho * A * u^{**2} / \text{total mass}$

```
'''
## WHAT CHANGES WITH TIME!?!?!?!?!?!? ##
'''
mass / (total mass - mass flow rate * time)
height / (velocity times time)
velocity / (acceleration times time)
acceleration / (forces)
density (height)
drag coeff (mach # -> velocity divided by acoustic speed (based off
atmospheric temperature vs height))
gravity (height)
area (kinda? with stage)
'''
##### Thrust #####
'''
sX_avgT/sX_instM

s1_instM = (p1 + s3 + s2 + (s1 - mdot*s1_t))
s2_instM = (p1 + s3 + (s2 - mdot*s2_t))
s3_instM = (p1 + (s3 - mdot*s3_t))
p1_instM = p1
'''
#### Drag ####
'''
drag = 0.5 * rho * CD * u**2 * A / sX_instM

s1_instM = (p1 + s3 + s2 + (s1 - mdot*s1_t))
s2_instM = (p1 + s3 + (s2 - mdot*s2_t))
s3_instM = (p1 + (s3 - mdot*s3_t))
p1_instM = p1
'''
#### Gravity ####
'''
g = G*ME / ((RE + h)**2)
'''
def grav(h):
    G = 6.674E-11 # m^3/kg*s^2
    ME = 5.972E24 # kg
```



```

    RE = 6.371E6 # m
    g = G*ME / ((RE + h)**2)
    return g

#### Drag Coefficient ####
'''
Figure 4-4 from textbook for 0 deg angle of attack
Interpolate for in betweens
'''
def calcCD(mach):
    mach_nums = np.array(np.linspace(0.2,5.4,27))
    cd_coeffs =
np.array([0.15,0.15,0.15,0.18,0.30,0.41,0.36,0.30,0.26,0.25,0.24,0.23,0.22
,0.21,0.21,0.20,0.20,0.19,0.18,0.17,0.17,0.16,0.16,0.15,0.15,0.15])

    drag_coeffs = np.column_stack((mach_nums, cd_coeffs))
    if mach == 0:
        cd = 0
    elif mach >= 0.2 and mach <= 5.4:
        cd = np.interp(mach, drag_coeffs[:,0], drag_coeffs[:,1])
    else:
        cd = 0.15

    return cd

#### Density & Temperature ####
def atmos(h):
    # Returns air temp T [K] and density rho [kg/m^3]
    # Dependent on altitude (h) in meters
    # https://www.grc.nasa.gov/www/k-12/airplane/atmosmet.html
    if h > 25000:
        T = 141.94 + 0.00299*h
        press = 2.488 * (T/216.6)**(-11.388)
    elif h > 11000 and h <= 25000:
        T = 216.69
        press = 22.65 * np.exp(1.73-0.000157*h)
    else:
        T = 288.19 - 0.00649*h
        press = 101.29 * (T/288.08)**5.256
    rho = press / (0.2869*(T))
    return [T,rho]
#### Mach # ####
'''
M = u / as (acoustic speed)

as = np.sqrt(R*Mass*T)
'''

```

```

def calcMach(T,u):
    R = 8314 # Universal Gas Constant
    airM = 28.96 # Molecular weight of air
    Rspe = R / airM
    gamma = 1.402 # Wolfram: Assume constant specific heat ratio
    mach = np.abs(u) / np.sqrt(gamma*(Rspe)*T)
    return mach

#####
#
massTotal = pl_mass + s3_massTotal + s2_massTotal + s1_massTotal # total
mass
masses_t = [massTotal] # array to track mass changing over timesteps

h_t = [0] # height / (velocity times time)
u_t = [0] # velocity / (acceleration times time)
''' UNUSED THUS FAR
a_t = [0] # acceleration / (forces)
rho_t = [] # density (height)
cd_t = [] # drag coeff (mach # -> velocity divided by acoustic speed
(based off atmospheric temperature vs height))
'''
# gravity (height)
# area (kinda? with stage)
#
#####
dt = 0.05 # time increment (0.05s)
burnTime = s1_tp + s2_tp + s3_tp # aka s3_sep
burnout_time = 976 # time to fall back down range...just a guess for now
timesteps = int((burnTime + burnout_time)/dt) # Right now I think this
will only bring us up to 3rd stage burnout (adding 2 will get to stage 3
sep)
time = np.linspace(0,burnTime + burnout_time,timesteps+1)

for i in range(timesteps):
    calcTime = round(i*dt,2) # calculates runtime (not a function)
    if calcTime <= s1_sep:
        # stage 1
        sX_avgT = s1_avgT
        sX_mDot = s1_mDot
        A = 0.25*np.pi*(s1_diam**2) # area calculated based off larger
motor diameter
        if calcTime == s1_sep: # Stage 1 Separation
            masses_t[-1] -= s1_massBurn
            print('Stage 1 Separation Complete')
        elif calcTime > s1_sep and calcTime <= (s2_sep):
            # stage 2

```

```

        sX_avgT = s2_avgT
        sX_mDot = s2_mDot
        A = 0.25*np.pi*(s2_diam**2)
        if calcTime == (s2_sep): # Stage 2 Separation
            masses_t[-1] -= s2_massBurn
            print('Stage 2 Separation Complete')
    elif calcTime > (s2_sep) and calcTime <= (s3_sep):
        # stage 3
        sX_avgT = s3_avgT
        sX_mDot = s3_mDot
        A = 0.25*np.pi*(s3_diam**2)
        if calcTime == (s3_sep): # Stage 3 Separation
            masses_t[-1] -= s3_massBurn
            print('Stage 3 Separation Complete')
    else:
        # burnout (no more thrust)
        sX_avgT = 0 # payload does not produce thrust
        sX_mDot = 0 # mass no longer changes
        masses_t[-1] = pl_mass # final mass change that ensures mass at
        burnout is only payload

    u = u_t[i]
    m = masses_t[i] - sX_mDot*dt # mass change due to propellant
    masses_t.append(m)
    h = h_t[i]

    g = grav(h)
    T, rho = atmos(h)

    mach = calcMach(T,u)
    cd = calcCD(mach)

    thrust_force = sX_avgT/m
    drag_force = (0.5*cd*rho*A*(u*np.abs(u))/m)

    a = thrust_force - g - drag_force
    new_u = u + a*dt
    new_h = h + u*dt
    u_t = np.append(u_t,new_u)
    h_t = np.append(h_t,new_h)

plt.plot(time,h_t)
plt.plot(s1_sep,h_t[int(s1_sep/dt)],'o', label = 'S1 Separation')
plt.plot((s2_sep),h_t[int((s2_sep)/dt)],'bo', label = 'S2 Separation')
plt.plot((s3_sep),h_t[int((s3_sep)/dt)],'ro', label = 'S3 Separation')
plt.plot(np.argmax(h_t)*dt,max(h_t),'k*', label = 'Max Altitude')
plt.legend()

```

```
plt.show()
```

References

[1] G. Sutton and O. Biblarz, Rocket Propulsion Elements. Hoboken, New Jersey: John Wiley & Sons, Inc. 2017

[2] <https://www.grc.nasa.gov/www/k-12/airplane/atmosmet.html>

